Factovia Technical Training

barbara

April 6th, 2026 – version 11.0

Index

1. Custom Apps Development
2. API Integrations

Custom App Development

Images vs Containers

Docker Fundamentals

💡 **Key idea:**

Image = what to run

Container = running app

🏠 **Analogy:**

Blueprint → House

Image 🧱	Container 🚀
Template (read-only)	Running instance
Built once	Created from image
Defines the app	Executes the app

Pulling Images from Repos

Docker Fundamentals

💡 **Key idea:**

Pull = download image from registry

Run = start container locally

📦 **Example flow:**

Docker Hub → Local Image → Running Container

Step	Command
Pull image	`docker pull nginx`
List images	`docker images`
Run container	`docker run -p 80:80 nginx`

Dockerfile

Docker Fundamentals

💡 **Key idea:**
Dockerfile = instructions to build an image

📦 **Flow:**
Dockerfile → Image → Container

Concept	Example
Base image	`FROM node:18`
Set workspace	`WORKDIR /app`
Copy files	`COPY . .`
Install deps	`RUN npm install`
Run app	`CMD ["npm","start"]`

Building images and pushing to repos

Docker Fundamentals

💡 **Key idea:**

Build = create image locally

Push = upload to registry

📦 **Flow:**

Code → Image → Registry

Step	Command
Build image	`docker build -t my-app:1.0 .`
Tag image	`docker tag my-app:1.0 repo/my-app:1.0`
Login to registry	`docker login`
Push image	`docker push repo/my-app:1.0`

Docker-Compose

Docker Fundamentals

💡 **Key idea:**
Compose = define & run multi-container apps

⚠️ **IMPORTANT (Barbara):**
Barbara Edge nodes use **docker-compose.yml** to deploy apps

📦 **Flow:**
docker-compose.yml → Multiple containers running together

Concept	Example
Define services	`services:`
App service	`app:`
Image / build	`image:` / `build:`
Ports	`ports: ["3000:3000"]`
Run all	`docker compose up`

Deploying a basic app using a image

Docker Fundamentals

💡 **Key idea:**
Run a ready-to-use app directly from a registry image

Example:

```
services:
  grafana:
    image: grafana/grafana
    ports:
      - "3000:3000"
```

📦 **Flow:**
Registry image → docker-compose.yml → Running app locally

Step	Example
Use image	`image: grafana/grafana`
Expose port	`ports: ["3000:3000"]`
Start service	`docker compose up -d`
Check app	Open `http://localhost:3000`

Deploying a basic app building the image

Docker Fundamentals

💡 **Key idea:**

Build image locally before running the app

Example:

```
services:
  app:
    build: .
    ports:
      - "3000:3000"
```

📦 **Flow:**

Code + Dockerfile → Image → Container running locally

Step	Example
Define build	`build: .`
Expose port	`ports: ["3000:3000"]`
Start service	`docker compose up -d`
Check app	`http://localhost:3000`

Global Secrets Usage (I)

Docker Fundamentals

💡 **Key idea:**
Global Secrets = secure, centralized config injected at runtime (no hardcoding)

Example (Grafana)

⚠️ **IMPORTANT (Barbara):**
Secrets are defined in Panel and injected as environment variables into the container

📦 **Flow:**
Global Secret → Env Vars → Container Configured Securely

Concept	Example
Global Secret	Managed sensitive value (Barbara)
Inject as env	`GF_SECURITY_ADMIN_USER`
Inject as env	`GF_SECURITY_ADMIN_PASSWORD`
Deploy	`docker compose up`

Global Secrets Usage (II)

Docker Fundamentals

💡 **Key idea:**
Secrets are injected at runtime inside the container (not exposed in host env)

Example (Ubuntu)

⚠️ **IMPORTANT (Barbara):**
Secrets are injected into the container but **NOT defined as environment variables in the Edge node (host)**

📦 **Flow:**
Global Secret → Container Env → Runtime Usage

Concept	Example
Global Secret	Managed sensitive value (Barbara)
Inject as env	`MY_SECRET`
Use in container	`echo $MY_SECRET`
Deploy	`docker compose up`

Global Secrets Usage (III)

Docker Fundamentals

💡 **Key idea:**
Secrets are consumed directly from code via environment variables

Example (Python - boilerplate)

⚠️ **IMPORTANT (Barbara):**
Secrets are injected into the container and accessed from code,
but **NOT available as env vars in the Edge node (host)**

📦 **Flow:**
Global Secret → Env Var → Python (`os.getenv`)

Concept	Example
Global Secret	Managed sensitive value (Barbara)
Inject as env	`MY_SECRET`
Access in code	`os.getenv("MY_SECRET")`
Deploy	`docker compose up`

App config & Global Config usage

Docker Fundamentals

💡 **Key idea:**
App Config and Global Config are defined in Panel and mounted inside the container as JSON files

Example (Python - boilerplate)

⚠️ **IMPORTANT (Barbara):**
Config is **not injected as env vars**
It is available through a **bind mount** in `/appconfig`

📦 **Flow:**
Config → `/appconfig/*.json` → Python `json.load()`

Concept	Example
App Config	`/appconfig/appconfig.json`
Global Config	`/appconfig/global.json`
Format	JSON file
Access in code	`json.load(...)`

Volumes usage

Docker Fundamentals

💡 **Key idea:**

Volumes persist data and decouple storage from containers

- **Bind mounts** → host path (dev/debug)
- **Internal volumes** → managed by Docker
- **External volumes** → pre-created & reusable

Example (Grafana)

⚠️ **IMPORTANT (Barbara):**

- Bind mounts expose host files
- Internal volumes are preferred for portability
- External volumes enable sharing data across apps

Type	Example
Bind mount	`./data:/data`
Internal volume	`data:/data`
External volume	`external_data:/data`
Purpose	Persistent storage

Networks

Docker Fundamentals

💡 **Key idea:**

Containers communicate using **service names as DNS**

Example (Grafana)

⚠️ **IMPORTANT (Barbara):**

- Services in the same network can communicate internally
- No need to expose ports for internal communication
- `barbaraServices` **network is always available**
- Use it to enable **inter-application communication (DNS-based)**

Concept	Example
Default network	Auto-created by Compose
Service access	`http://service_name:port`
Define network	`networks:`
Attach service	`networks: [app-net]`

Resource Management

Docker Fundamentals

💡 **Key idea:**
Limit and reserve resources to ensure stable execution on Edge nodes

⚠️ **IMPORTANT (Barbara):**

- Resources are **limited per app** in Edge nodes
- Avoid overconsumption → ensures **multi-app stability**
- Define both **limits and reservations** when possible

📦 **Flow:**
Config → Resource limits → Controlled container usage

Resource	Example
CPU limit	`cpus: "0.5"`
Memory limit	`mem_limit: 512m`
Reservations	`mem_reservation: 256m`
Define in svc	`deploy/resources`

Accessing Devices

Docker Fundamentals

💡 **Key idea:**

Containers access hardware via **device mapping + proper permissions**

⚠️ **IMPORTANT (Barbara):**

- Devices must exist on the Edge node
- `privileged: true` **is NOT allowed**
- Use `group_add` (e.g. `dialout`, `video`) for access

📦 **Flow:**

Host device → Permissions (groups) → Container access → App usage

Concept	Example
Device mapping	`/dev/ttyUSB0`
Add to container	`devices:`
Access in app	Read/write from device file
Use case	IoT / hardware integration

Restart Policies

Docker Fundamentals

💡 **Key idea:**
Restart policies control container lifecycle and recovery behavior

⚠️ **IMPORTANT (Barbara):**

- `restart: always` is **FORBIDDEN**
- Use `on-failure` or `unless-stopped`
- Ensures controlled and safe recovery on Edge nodes

📦 **Flow:**
Failure → Restart policy → Container recovery

Policy	Example
No restart	`restart: "no"`
On failure	`restart: on-failure`
Unless stopped	`restart: unless-stopped`
Always ❌	`restart: always` (forbidden in Barbara)

Docker Limitations

Docker Fundamentals

💡 **Key idea:**
Barbara enforces strict rules for **security, persistence, and portability**

⚠️ **IMPORTANT (Barbara):**

- Other files in ZIP are **overwritten on update**
- Use `persist/` for durable data
- Host access outside allowed paths **IS FORBIDDEN.**

Limitation	Rule
Compose filename	`docker-compose.yml` / `.yaml` only
Persistence path	Must be inside `./persist/`
Bind mounts	Only `./persist/`, `./appconfig`, `./sys/`
Volumes	Native volumes allowed
Privileged mode	❌ Not allowed
Capabilities	❌ No admin capabilities

Docker Compose Limitations

Docker Fundamentals

💡 **Key idea:**

Barbara enforces stricter Compose validation than standard Docker

⚠️ **IMPORTANT (Barbara):**

- `restart: no` → ❌ invalid (must be `"no"`)
- `name`, `tty` → **not allowed**
- `env_file` must be defined as a **list**
- Misconfiguration → app will **not deploy**

📦 **Flow:**

docker-compose.yml → Validation rules → Deployment or failure

Limitation	Rule
Restart policy	`restart: "no"` (quotes required)
Name property	❌ Do NOT use `name:`
`tty`	❌ Not supported
`env_file`	Must use list format (`- .env`)
Syntax strictness	Must follow supported Compose spec
Validation	Invalid configs → deployment failure

API Integrations

VSCode Extension

API Access

💡 **Key idea:**

All-in-one tool to **develop + deploy Barbara apps from VSCode**

⬇️ **Download:**

Install Barbara VSCode Extension

🔐 **Required credentials:**

- `Client ID` *(Enterprise)*
- `Client Secret` *(Enterprise)*
- `Barbara User`
- `Barbara Password`

Feature	Description
Dev & deploy	Build, upload & deploy apps
Boilerplates	Python / Node.js templates
App Deployment	Deploy to Edge nodes

Python SDK

API Access

💡 **Key idea:**

Deploy and manage AI Models on Barbara **directly from Python code**

⬇️ **Install (PyPI):**

https://pypi.org/project/barbara-sdk/

Example:

```
import barbara

bbr = barbara.ApiClient('client_id',
'client_secret', 'username', 'password')
bbr.models.list()
```

Feature	Description
Model upload	Upload models from Python
Deployment	Deploy models to Edge nodes
Automation	Automate ML workflows
Integration	Use in scripts or Jupyter notebooks

Github CI/CD Pipelines

API Access

REPO: https://github.com/Barbaraedge/barbara_github_actions_sample

💡 **Key idea:**
Full CI/CD pipeline automates **build** → **deploy** → **verify** → **release**

⚙️ **Trigger:**

```
git tag v0.2.0
git push --tags
```

🔐 **Required configuration:**

- **Repo variables:** app name, registry, architectures
- **Secrets:** registry credentials
- **Barbara env:** API URL, Keycloak, user credentials

⚠️ **IMPORTANT (Barbara):**

- Deployment fails early if targets don't exist
- Health check with **auto-rollback**
- Only CI-built images are valid for deployment

📦 **Flow:**
Git tag → CI pipeline → Image + ZIP → Barbara → Devices 🚀

barbara

The Edge AI Platform

Barbara
@barbara_edge_ai
@Barbara-edge
https://barbara.tech
hola@barbara.tech